FALL FEST | QUANTUM COMMUNICATION & NETWORKING

Q-NetOps

Autonomous Quantum-Network Controller for Fidelity-Aware Routing and Resource Scheduling

DETAILED PROJECT BRIEF

A teammate-ready technical narrative for a judge-facing presentation and report: problem statement, proposed solution, architecture, simulator, controller policies, data and tools, evidence, limitations and future scope.

65 / 200	0.769	91.4 ms	16.4%
Adaptive qualified deliveries	Delivered-pair mean fidelity	Mean delivery time	Mean memory occupancy

Evidence label: headline values are from one seeded simulation mission (Photon-loss spike, seed 7, 200 requests per policy). They are simulated outcomes, not measurements from a deployed repeater network or quantum processor. This brief separates outcomes, estimates, external experimental inputs and proposed future work.

PPT positioning line

“Q-NetOps tests whether an adaptive controller can make better connection decisions when quantum links are probabilistic, memories are finite, and entanglement quality decays while the network waits.”

01 | Executive summary

Q-NetOps is a research prototype for comparing routing and scheduling policies in a simulated quantum repeater network. It exposes core physical constraints in software: link generation can fail, stored pairs lose fidelity, memories are finite, and entanglement swaps are probabilistic. Its adaptive policy combines a read-only network snapshot, named route scores and repeated rollouts through the same event simulator to rank candidate paths and select an action.

The strongest contribution is methodological: instead of choosing a path only by distance, the prototype evaluates delivery probability, fidelity, latency, memory headroom, coherence lifetime and recovery exposure, then records request outcomes and controller events for replay and policy comparison. It is an interpretable model-predictive heuristic, not a trained AI model.

Problem statement (PS)

Long-distance quantum communication depends on distributing entangled pairs across repeater chains. Fiber attenuation reduces photon transmission success as distance grows. A generated pair can decohere while waiting in memory. Intermediate swaps are imperfect and probabilistic. Classical shortest-path routing typically optimizes path cost while ignoring some combination of quantum fidelity thresholds, deadlines, finite memories and contention from overlapping requests.

The engineering gap is coordinating these constraints under uncertainty: poor choices can waste attempts, retain aging pairs, or produce a final state below the application quality requirement. Q-NetOps makes this a reproducible simulator and policy-comparison problem.

Research question

Under the same topology, traffic, noise, memory limits, random seed and deadline, do fidelity- and resource-aware decisions improve qualified end-to-end delivery versus a static shortest-path baseline? What trade-offs appear in latency, memory use, retries and recovery?

Prototype success criteria

- A repeatable seeded simulator with per-request outcomes and recorded decisions.
- Side-by-side policy comparison using a common configuration and explicit metric denominators.
- A route choice that considers fidelity feasibility and current resource headroom, not path length alone.
- An auditable debrief and exportable report that separates observed outcomes from controller estimates.

02 | Problem model and quantum-network concepts

The network is represented as a graph: vertices are endpoints or repeaters, and edges are optical links. A request asks the controller to establish an end-to-end entangled pair. Candidate routes differ in physical quality, timing and resource demands.

Constraint	Why shortest path misses it	Q-NetOps treatment
Photon loss	One long or noisy segment can dominate chain success.	Distance-dependent link success, link noise and repeated attempts.
Memory decoherence	Stored pairs lose quality while waiting for other segments.	Werner-state memory decay, coherence and purge thresholds.
Imperfect swapping	Each intermediate operation adds probabilistic failure.	Swap success probability and sequential swap fidelity.
Finite memory	Overlapping requests compete for repeater and endpoint slots.	Bounded slots, occupancy, expiry/purge and contention.
Deadline and target	A pair can arrive too late or below required quality.	Request deadline and fidelity qualification.
Changing state	Current memory availability changes as requests progress.	Adaptive snapshot and event-driven route decisions.

Key terms for the presentation

Term	Meaning in this project
Entangled pair	A shared two-qubit state distributed between nodes.
Fidelity	Overlap with the intended target state; higher means closer to ideal.
Werner state	Simplified noise model describing an imperfect pair with one fidelity value.
Entanglement swapping	Repeater operation that joins neighboring entangled segments.
Coherence time	Characteristic time scale over which a stored state loses quality.
Qualified delivery	Request marked delivered and meeting its configured fidelity threshold.
Digital-twin snapshot	Read-only copy of simulated resources used to test route feasibility and scores.
Event trace	Recorded simulator milestones that can be replayed and analyzed.

Important scope boundary

The map includes illustrative corridors across India and ocean regions; these do not claim real quantum infrastructure. The primary mission simulation uses a small synthetic Chennai-region repeater graph. Mission outcomes and forecasts come from software simulation.

03 | Proposed solution and controller loop

Mission configuration defines traffic pressure, noise, fidelity threshold, memory capacity, retry budget, seed and deadline. The simulator evolves network state and emits events. Candidate routes are assessed against a frozen snapshot and by seeded rollouts. The selected controller acts; the system records outcomes and compares policies.

1 Mission request + configuration	2 Shared-network event simulator
3 Snapshot of memory/link state	4 Candidate paths + feasibility gates
5 64-rollout forecast + route score	6 CONNECT / WAIT / RETRY / REROUTE
7 Event trace + request outcomes	8 KPI summary, debrief + report

How the adaptive choice works

- Enumerate candidate paths and capture available slots in a deep-copied network snapshot.
- Estimate link success, swap success, path fidelity, latency, memory headroom, coherence lifetime and recovery exposure. Feasibility gates include target fidelity, deadline, memory and lifetime.
- Run 64 deterministic single-request rollouts per candidate through the same event simulator. Blend the named weighted objective (60%) and rollout delivery rate (40%) for route ranking. These are modeled estimates, not telemetry.
- Choose CONNECT when a route is currently feasible and forecasts delivery; WAIT when memory is the limiting constraint or a viable attempt window is expected; REROUTE when constraints fail. RETRY is used within the connection lifecycle after failures.
- Record selected path, action, reason, candidate evaluation and resulting events where available; aggregate policy outcomes after requests complete.

Selective repair and shared-resource scheduling

When the adaptive traffic controller repairs a route, it intersects old and new links. Entangled pairs and in-flight attempts associated with links retained on the new route can be preserved; work tied to removed links is released or canceled. A route-repaired event records the change and preserved-pair count. This is simulator behavior, subject to timing, memory and route constraints, not a deployed network interface.

Requests overlap on a persistent benchmark network. Links serialize generation attempts, successful pairs reserve endpoint memory until swap, purge or expiry. Adaptive scheduling prioritizes least estimated deadline slack, then favors the request with more of its route already entangled. Classical baselines use first-come-first-served arbitration.

04 | Architecture and data flow

Layer / module	Responsibilities	Outputs / interfaces
Presentation - app.py	Streamlit mission controls, illustrative maps, Policy Arena, physics, IBM hardware, chat, event replay, debrief and report.	Controls, charts, tables, event views, local downloads.
API - q_netops/api.py	Local FastAPI service: health, corridor concepts, deterministic chat, bounded simulation.	JSON at /api/health, /api/corridors, /api/chat, /api/simulate; Swagger /docs.
Controllers	Static Dijkstra, greedy link quality, Q-NetOps adaptive policy.	Action, chosen path, reason, candidate scores.
Digital twin	Read-only simulated network snapshot; feasibility and auditable route-risk scoring.	Constraint flags, score, projected fidelity, headroom, modeled risk.
Simulation engine	Overlapping requests, link attempts, stored-pair expiry, swaps, route repair and traces.	Per-request outcomes and event logs.
Physics / network / memory	Attenuation, Werner decay, swapping, synthetic graph, bounded slot banks.	Probabilities, state values and mutable simulated network.
Metrics evaluator	Policy-level delivery, fidelity, latency, memory, expiry and risk aggregates.	Pandas summary for UI, API, charts and reporting.
Assistant / analyst	Intent-matched local helper uses provided context; analysis/report uses mission result data.	Deterministic responses, debrief and PDF.
IBM integration	Optional Qiskit IBM Runtime status and Bell-state tomography workload.	QPU job/tomography result if user configures credentials and submits.

End-to-end data lineage

User controls -> validated SimulationConfig -> benchmark run for each controller -> request rows and event records -> metrics evaluator -> Streamlit charts/debrief -> PDF or JSON/CSV export. The local API translates validated fields into the same simulation configuration and returns policy summaries and request outcomes. Ocean map overlays do not feed the simulator.

API contract at a glance

Method / path	Purpose	Boundary
GET /api/health	Service health	Status and service identifier.
GET /api/corridors	Illustrative corridors	Explicitly not live infrastructure.
POST /api/chat	Local help	Deterministic assistant; no external LLM call.
POST /api/simulate	Bounded comparison	Validates request count 1-100, seed, fidelity, noise, traffic, memory, optional fault, deadline, retries and route weights; returns policy summaries, request outcomes and injected-fault metadata.

05 | Simulator, physics and controller policies

For each policy, benchmark traffic runs on a persistent network. Seeded request arrivals overlap. A link performs one generation attempt at a time; propagation delay is modeled. Successful elementary pairs reserve endpoint memory and can decay, be purged, swapped or released at request termination. Policies are run from the configured conditions for comparative evaluation.

Core equations (as implemented)

Model	Equation / rule	Interpretation
Link generation	p(L)=clamp(eta * 10^(-alpha L / 10),0,1)	Source efficiency eta times attenuation over distance L.
Werner memory decay	F(t)=0.25+(F0-0.25) exp(-t/Tcoh)	Stored pair fidelity approaches random Werner floor 0.25.
Two-pair swap	Fswap=F1 F2+((1-F1)(1-F2))/3	Conditional output fidelity; separate swap probability affects delivery chance.
Path projection	Decay each link for holding time; sequentially combine with swap_fidelity.	Candidate projection; event simulation determines actual outcomes.
Adaptive risk	1 - rollout delivery rate when rollouts are available.	Modeled estimate including represented retry, expiry, swap and deadline behavior.

Mission configuration knobs

Input	Effect
Scenario / stress mode	Select baseline scenario and noise/stress conditions.
Seed	Reproducibility for stochastic draws and arrivals.
Requests per policy	Benchmark sample size; larger counts add runtime.
Target fidelity	Minimum output quality for qualified delivery.
Noise / calibration	Synthetic noise scaling or optional experiment-derived proxy.
Traffic load	Overlapping request arrival pressure and contention.
Memory slots	Finite storage capacity at each repeater.
Retry budget / deadline	Bounds attempt lifecycle and request time.
Route weights	Relative reliability, fidelity, latency, resources and recovery priorities.
Optional degraded link	Inject a named synthetic link fault for a stress test.

Three policies

Policy	Behavior	Trade-off
Static Dijkstra	Fixed route choice; FCFS arbitration.	Simple and cheap; may overlook fidelity and current headroom.
Greedy link quality	Route choice based on local link quality.	May prefer strong edges without full chain-level constraint view.
Q-NetOps adaptive	Snapshot gates, weighted route score, 64-rollout estimate, shared-state scheduling and limited repair.	Richer evidence at extra compute cost; rollout is not full future-queue replay.

06 | Data, resources and tools used

The project combines public experimental calibration data, precomputed simulation results, local assets and an optional cloud quantum-computing integration. Keep these evidence classes distinct in the presentation.

Resource	Usage	Factual boundary
Aliro BBM92 CSV (6 runs)	Measured QBER and link metrics can calibrate elementary-link Werner fidelity via F=1-1.5 x QBER.	Published experiment data; not repeater deployment or live telemetry. Source: github.com/aliro-technologies/realistic-quantum-network-simulation-for-bbm92.
Robustness benchmark	12,000 request-level simulation trials and 12 summary rows described by four scenarios x 10 seeds x 100 requests per policy.	Precomputed simulation evidence; regenerate with documented script/config.
Synthetic Chennai graph	Small repeater topology for main mission simulation.	Synthetic configured topology; not surveyed physical fiber.
Illustrative corridor map	India, Indian Ocean, Bay of Bengal/Andaman, Pacific and Atlantic concepts.	Map-only geography; does not assert operational quantum links.
IBM Quantum / Qiskit Runtime	Optional QPU status and Bell-state preparation with X/Y/Z tomography.	Requires account/key. Tests circuit execution, not repeater telemetry. No hardware job is claimed here.
SeQUeNCe / NIST MQNS	Public simulators referenced as design context.	Inspiration only; no project source copied. Check licenses before reuse.

Software stack and local versions

Package	Role	Requirement / verified version
Python	Runtime	3.13
Streamlit	Dashboard	>=1.40 / 1.66.0
Pandas	Tables and metrics	>=2.2 / 3.0.6
Plotly	Interactive charts	>=5.24 / 7.1.0
Folium + streamlit-folium	Leaflet map	>=0.17, >=0.24 / 0.20.0, 0.27.4
FastAPI + Uvicorn	Local API	>=0.115, >=0.30 / 0.141.1, 0.54.0
Qiskit + IBM Runtime	Optional circuit/hardware	>=2.5.2, >=0.47 / 2.6.0, 0.50.0
ReportLab / pypdf	PDF generation / validation	Available in workspace runtime.

Map tiles

Folium renders Leaflet maps; the standard OpenStreetMap tile layer requires internet access and attribution. The simulator does not require live map data. For public deployment, check tile-provider usage conditions and change providers if traffic grows.

07 | Evaluation evidence and observed outcomes

A fair policy comparison uses identical topology, configuration, request count and seed. The dashboard values below are a single seeded demo mission, not evidence of universal improvement. Use the multi-seed benchmark and its confidence intervals for broader claims, naming the exact data and execution.

Current mission configuration

Field	Value
Scenario	Photon-loss spike
Seed	7
Requests per policy	200
Target fidelity	0.70
Traffic pressure	0.15
Memory slots per repeater	10
Retry budget	12
Deadline	100 ms
Injected fault	None

Policy Arena snapshot

Metric	Adaptive	Static baseline / UI delta	Interpretation
Qualified deliveries	65/200 = 32.5%	10/200 = 5.0% (UI delta: +27.5 pp)	Run-level fidelity-qualified rate.
Mean fidelity, all outcomes	0.558	0.415 (UI delta: +0.143)	Includes unsuccessful outcomes.
Mean fidelity, delivered pairs	0.769	Unavailable in captured evidence	Only delivered pairs.
Mean delivery time	91.4 ms	99.8 ms (UI delta: -8.4 ms)	Per-policy mean for this run.
Mean memory occupancy	16.4%	26.8% (UI delta: 38.8% lower)	UI reports relative reduction.
Deadline misses	135	Unavailable in captured evidence	Use trace reasons; do not infer causes.

Metric definition: qualified delivery rate

Qualified delivery rate is the share of requests marked delivered whose final fidelity met the configured target. At target 0.70, 65/200 = 32.5%. Delivered-pair fidelity averages only delivered pairs (0.769); mean fidelity across outcomes (0.558) includes failed or unqualified outcomes. These denominators are not interchangeable.

Verification evidence

- 18 unit tests previously passed, covering API contracts/validation, simulator and event records, deterministic assistant, grounded analysis, baseline deltas and PDF payload/content.
- Local API smoke run: seed 17, 2 requests per policy, three policy summaries, 6 outcomes/traces, HTTP 200. This validates the interface only, not statistical performance.
- Health, corridors, chat and simulate routes were verified through local Swagger. API has no authentication or persistent storage.
- No IBM QPU job was run for this evidence snapshot.

08 | Limitations, risks and future scope

A technically credible presentation says what the prototype does not prove. These roadmap items are proposed future work and are not current features unless described above.

Boundary	Why it matters	Future improvement
Synthetic small topology	Cannot establish field performance.	Validate on testbed/reference topology; version physical parameters.
Forecast ignores future full queue	Snapshot gates present resources, but candidate forecast is isolated single-request rollout.	Queue-aware predictive horizon jointly simulating pending requests and releases.
Heuristic, not trained ML	No trained policy or train/test evaluation.	Collect controlled data; compare heuristic, optimization and learned policy on held-out seeds with explainable reasons.
Limited failure diversity	No node outages, urgency classes, cancellation or starvation guarantees.	Add configurable outages, service classes, cancellation and fairness; measure tail latency.
Calibration uncertainty	Werner proxy and six-run data simplify diverse devices/channels.	Represent parameter ranges, sensitivity and richer noise channels; validate against measured traces.
Local API and runtime	No auth/persistence; synchronous simulation can be slow.	Async jobs/progress/cancel, auth, rate limits, audit and observability.
No durable mission history	Results may not survive process/session lifecycle.	Store provenance, run manifests and replayable results.
IBM scope	Tomography is circuit-level, not network-level.	Use QPU results only for submodel calibration; clearly separate from network claims.
Ocean map is conceptual	May be mistaken for deployed infrastructure.	Label routes as concepts; document geography and distinguish fiber from quantum links.

Prioritized roadmap

Horizon	Increment	Acceptance signal
Next sprint	Async runs with run IDs/progress; persist config/results; queue-aware forecast as separate benchmarked feature.	Responsive UI, deterministic replay and paired-seed comparison.
Near term	Failure taxonomy, fairness classes, stress suite, sensitivity and confidence intervals.	Report samples, interval method, deltas, trace-backed causes.
Research	Testbed data, calibration uncertainty, external validation.	Trace-to-model validation and error bars.
Deployment	Identity/access, job queue, monitoring, retention and operational policies.	Threat model, load test, access controls and documented SLO.

Risks to manage

- Metric confusion: state denominator and whether each number is measured, simulated, estimated or predicted.
- Demo runtime: adaptive candidate rollouts cost compute; use a smaller live interactive run and precomputed robustness evidence.
- Credentials: keep IBM secrets in ignored local configuration or backend secret storage; exclude them from documents and source control.
- Evidence scope: benchmark outcomes support claims only within the simulated assumptions and tested scenarios.

09 | Suggested PPT structure and judge demo

A 10-slide narrative from engineering need to evidence and validation:

Slide	Title	Presenter focus / visual
1	Q-NetOps: smarter decisions for quantum links	One-line pitch; route/map labeled synthetic.
2	Why classical routing falls short	Attenuation, decoherence, probabilistic swaps, finite memory.
3	Problem statement and research question	Define request, target and qualified delivery.
4	Architecture	Simulator/controller/UI/API flow.
5	Physics in the model	Equations and link-memory-swap illustration.
6	How adaptive control chooses	Snapshot, gates, score, 64 rollouts and reason; say heuristic.
7	Mission control	Photon-loss spike, seed 7, 200 requests, target 0.70.
8	Results versus baselines	Qualified delivery and deltas; state single seed and denominator.
9	Limitations and validation	Synthetic topology, conceptual oceans, no QPU claim.
10	Roadmap and impact	Queue-aware model, multi-seed evidence, testbed, security.

90-second demo script

“We start with a request requiring fidelity of at least 0.70 before 100 milliseconds. The static baseline chooses a fixed route. Q-NetOps inspects available memory and candidate constraints, then repeatedly runs the same simulator to estimate alternatives. As requests overlap, links and memories contend. The event trace shows what the controller decided and what happened. In this seed-7 photon-loss mission, 65 of 200 adaptive requests qualified, or 32.5%; delivered-pair fidelity averaged 0.769. That is one simulation run, not field data. We are measuring multi-seed robustness and extending queue-aware forecasts before testbed validation.”

Live demo sequence

- Show mission configuration and state seed, fidelity threshold, deadline.
- Run and point to qualified delivery; distinguish overall mean fidelity.
- Open a request trace; narrate only events and reason fields actually recorded.
- Compare adaptive, Static Dijkstra and greedy; say whether single run or aggregate.
- Ask the deterministic local helper about a route only when trace context supports the answer.
- Download the mission report; close with limitations and next validation step.

Likely judge questions

Question	Factual answer
Connected to a real quantum network?	No. Main repeater topology and outcomes are synthetic simulation.
Is controller ML?	No. It is an interpretable heuristic/model-predictive policy with rollout estimates.
What is live in IBM tab?	If configured, QPU status and Bell-state tomography circuit job; not repeater telemetry. No job claimed here.
Where is training data?	There is no trained model. Six BBM92 experiment rows inform a fidelity proxy; benchmarks are simulated.
What is novel?	Explainable control combining physical constraints, shared memory, rollout-based evaluation, event logging and selective repair. Avoid a “first-ever” claim.
How validate next?	Multi-seed intervals, sensitivity, calibration, then compare against testbed traces.

10 | Reproducibility, runbook and references

Reproducibility checklist

- Record code version, scenario, seed, request count, configuration, injected fault and run date.
- Use identical configuration and seed across policies; separate single-run demo from aggregate evidence.
- Export request-level outcomes and traces with summaries.
- For controller changes, preserve previous configuration and compare paired seeds.
- Label calibration source and transformation; do not call proxy live hardware measurement.
- Keep secrets out of artifacts, screenshots, logs and source control.

Local run commands

Purpose	Command
Install	python -m venv .venv; .\.venv\Scripts\Activate.ps1; pip install -r requirements.txt
Dashboard	streamlit run app.py
Backend API	uvicorn q_netops.api:app --host 127.0.0.1 --port 8000
API docs	http://127.0.0.1:8000/docs
Unit tests	python -m unittest discover -s tests -v
Robustness benchmark	python scripts/run_robustness_benchmark.py --requests 100 --seeds 10 --first-seed 21

References and resource links

1. Aliro Technologies BBM92 simulator/data: https://github.com/aliro-technologies/realistic-quantum-network-simulation-for-bbm92
2. SeQUeNCe quantum network simulator: https://github.com/sequence-toolbox/SeQUeNCe (reference/inspiration; no source code copied).
3. NIST MQNS: https://github.com/usnistgov/mqns (reference/inspiration; review GPLv3 before reuse).
4. IBM Quantum / Qiskit Runtime: https://quantum.ibm.com/ and https://github.com/Qiskit/qiskit-ibm-runtime
5. FastAPI: https://fastapi.tiangolo.com/ ; Streamlit: https://docs.streamlit.io/ ; Qiskit: https://docs.quantum.ibm.com/
6. OpenStreetMap: https://www.openstreetmap.org/ (internet tiles; provide attribution and respect usage policy).

Final presentation takeaway

Q-NetOps is a transparent sandbox for asking a practical control question: when fidelity, time and memory all matter, can an adaptive controller outperform a fixed route? The current answer is promising in selected simulations and still needs broader statistical evaluation and hardware/testbed validation.

Teammate handoff: verify benchmark files and the final live-demo configuration before moving numeric results into a public PPT.